

Software Engineering Internship Challenge

SupportDesk — Mini Customer Support Ticket System

Objective

Build a small full-stack application that allows a company to create, organize, and manage customer support tickets.

This assignment will evaluate:

- Programming fundamentals
- Analytical thinking
- Problem-solving ability
- Frontend and backend understanding
- Database design
- Code readability
- Validation and error handling
- Initiative and product thinking
- Ability to explain technical decisions

The application does not need advanced visual design. A simple, clean, and functional interface is sufficient.

Time Limit

You should spend no more than **8 hours** on this challenge.

A complete basic solution is better than a large incomplete solution.

If you cannot complete everything within the time limit, submit what you have and clearly explain:

- What you completed
- What remains incomplete
- What problems you faced
- What you would implement next

Your prioritization and explanation will also be evaluated.

Business Scenario

A software company receives support requests from customers.

The support team needs a simple system where it can:

1. Create support tickets
2. View all tickets
3. Search and filter tickets
4. Change ticket status
5. Identify urgent tickets
6. Review basic ticket statistics

Functional Requirements

1. Create a Support Ticket

Create a form where a user can submit a support ticket.

Each ticket should contain:

- Customer name
- Customer email
- Subject
- Description
- Priority
- Status
- Created date
- Updated date

Available priority values:

- Low
- Medium
- High

The default status should be:

Open

Available status values:

- Open
 - In Progress
 - Resolved
-

2. Input Validation

The application must validate ticket data.

At minimum:

- Customer name is required
- Customer email is required
- Email must be in a valid format
- Subject is required
- Description must contain at least 10 characters
- Priority must be Low, Medium, or High

Backend validation is required.

The frontend should also display understandable error messages where possible.

3. Ticket Listing

Display all tickets in a table, card list, or another organized layout.

Each ticket should display:

- Ticket ID
- Customer name
- Subject
- Priority
- Status
- Created date

Users should be able to:

- Search by customer name, email, or subject
 - Filter by priority
 - Filter by status
 - Sort by creation date
-

4. Ticket Detail

Users should be able to open an individual ticket and view:

- Complete customer information
- Subject
- Description

- Priority
- Current status
- Created date
- Updated date

The user should also be able to update the ticket status.

5. Status Update

The user should be able to change a ticket's status between:

- Open
- In Progress
- Resolved

Whenever the status changes, the ticket's updated date should also change.

The backend should validate that only supported status values are accepted.

6. Urgent Ticket Detection

The system should automatically mark a ticket as urgent when either of the following conditions is true:

- The priority is High
- The description contains the word "urgent"

The word matching should not be case-sensitive.

Urgent tickets should be clearly identified in the interface.

This logic should be handled by the backend.

7. Dashboard Statistics

Display the following information:

- Total number of tickets
- Number of open tickets
- Number of tickets in progress
- Number of resolved tickets
- Number of urgent tickets

The statistics should be calculated using backend data.

Technical Requirements

The solution must include:

- A frontend application
- A backend API
- Persistent database storage
- Input validation
- Error handling
- Clear setup instructions
- At least two automated tests

The tests should cover important logic such as:

- Urgent-ticket detection
- Input validation
- Status updates
- Invalid status handling

User authentication is not required.

Suggested Technology

Preferred technologies include:

- JavaScript or TypeScript
- React or Next.js
- Node.js with Express or another backend framework
- SQLite, PostgreSQL, MySQL, or MongoDB

Candidates may use another programming language or framework if they can explain their decision.

SQLite is acceptable and recommended for easier local setup.

Suggested API Structure

A possible API structure is:

```
POST  /api/tickets
GET   /api/tickets
GET   /api/tickets/:id
```

```
PATCH /api/tickets/:id
PATCH /api/tickets/:id/status
GET    /api/dashboard
```

You may use a different structure if it is logical and documented.

Problem-Solving Requirement

The following situation must be handled:

A user submits a ticket with an email address that already exists in another ticket.

You must decide whether the system should:

- Allow the new ticket
- Block the new ticket
- Display a warning
- Link it with previous tickets from the same customer

There is no single correct answer.

Explain your decision in the README and describe the benefits and disadvantages of your approach.

Initiative Requirement

After completing the basic requirements, add **one useful improvement** of your own choice.

Possible improvements include:

- Ticket comments
- Assigning tickets to team members
- Pagination
- Dark mode
- Better mobile layout
- Customer ticket history
- Exporting tickets to CSV
- Priority-based sorting
- API documentation
- Better test coverage
- Docker setup
- Activity history
- Automatic ticket reference numbers

Your improvement should solve a real problem.

In the README, explain:

1. What you added
2. Why you selected it
3. What problem it solves
4. What you would improve further

Submission Requirements

1. Git Repository

Submit a Git repository containing the project.

Use multiple meaningful commits where possible.

Examples of useful commit messages:

```
Set up backend and database
Add ticket creation API
Add validation and error handling
Build ticket listing screen
Add urgent ticket detection
Add automated tests
```

Avoid submitting the complete application in one final commit.

2. README File

The README should include:

- Project overview
- Technology stack
- Setup instructions
- Database setup
- How to run the frontend
- How to run the backend
- How to run the tests
- API endpoint summary
- Assumptions made
- Duplicate email decision

- Initiative feature explanation
 - Known limitations
 - What you would build next
-

3. Time Log

Include an approximate time breakdown.

Example:

```
Planning: 30 minutes
Backend and database: 2 hours
Frontend: 2 hours 30 minutes
Testing: 45 minutes
Documentation and improvements: 45 minutes
Total: 6 hours 30 minutes
```

The time log does not need to be exact.

4. Demonstration

Provide either:

- A deployed application, or
- A 2–4 minute screen recording

Deployment is optional.

AI Assistance Policy

This challenge must be completed without generative AI assistance.

You may not use:

- ChatGPT
- Claude
- Gemini
- GitHub Copilot
- Cursor AI
- AI code generators
- AI debugging tools

- AI-generated tests
- AI-generated documentation
- AI features inside code editors

You may use:

- Official documentation
- Search engines
- Stack Overflow
- Framework documentation
- Library documentation
- Personal notes
- Normal IDE autocomplete without generative AI
- Standard project initialization tools

Any copied code, starter templates, or external references must be mentioned in the README.

Add the following declaration to the README:

I confirm that I completed this challenge without using generative AI, an AI coding assistant, or an AI-enabled editor. I understand the submitted code and can explain and modify it.

Important Notes

- Focus on functionality before design.
- Do not over-engineer the application.
- Write code that another developer can understand.
- Handle errors instead of assuming everything will work.
- Clearly document incomplete sections.
- You must be able to explain all submitted code.
- You may be asked to modify part of the application during the interview.